

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:
Мищенко Максим Юрьевич

Группа
К3339

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Описать все функциональные и нефункциональные требования к бэкенд-приложению
2. Выбрать формат реализации API, которого будем придерживаться: REST API или Backend For Frontend
3. Спроектировать все эндпоинты API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Описать тело каждого запроса и тело каждого ответа, продумать возможные ошибки, возвращаемые из API

Ход работы

1. Функциональные требования

1.1. Управление пользователями (Users)

- Регистрация — создание учётной записи с email, паролем, ФИО, телефоном и ролью (guest по умолчанию). Email должен быть уникальным.
- Аутентификация — вход по email и паролю с выдачей пары токенов (access JWT + refresh JWT).
- Просмотр профиля — получение информации о текущем аутентифицированном пользователе (без пароля).
- Редактирование профиля — изменение ФИО, телефона, пароля.
- Смена роли (только для администратора) — изменение роли пользователя (guest ↔ admin).

1.2. Управление ресторанами (Restaurants)

- Создание ресторана — администратор может добавить новый ресторан с названием, описанием, городом, адресом, кухней, средним чеком и геокоординатами (опционально).
- Редактирование ресторана — администратор может изменить любые поля ресторана.
- Архивация / восстановление — администратор может перевести ресторан в статус archived (скрыть из поиска) или обратно в active.
- Удаление ресторана — администратор может удалить ресторан (каскадно удаляются фотографии, столики и позиции меню).
- Поиск ресторанов — с фильтрацией по городу, кухне, среднему чеку (min/max), пагинацией.
- Просмотр деталей ресторана — полная информация + список фотографий + главная фотография + меню + отзывы.

1.3. Управление фотографиями (RestaurantImages)

- Загрузка фотографий — администратор может добавить несколько изображений к ресторану.
- Установка главного фото — администратор может отметить одно из изображений как основное (`is_main = true`). При этом предыдущее главное фото сбрасывается.
- Удаление фотографии — администратор может удалить любое фото ресторана.

1.4. Управление столиками (Tables)

- Добавление столика — администратор может добавить столик с указанием вместимости и метки (например, «Стол №4 у окна»).
- Редактирование столика — изменение вместимости и метки.
- Удаление столика — администратор может удалить столик при отсутствии активных бронирований.
- Просмотр столиков ресторана — список всех столиков с их параметрами.

1.5. Меню (MenuItems)

- Создание позиции — администратор может добавить блюдо с названием, описанием, ценой и категорией.
- Редактирование и удаление позиций меню.
- Просмотр меню ресторана — список всех блюд, сгруппированных по категориям.

1.6. Бронирования (Bookings)

- Создание бронирования — авторизованный гость выбирает ресторан, столик, дату, время начала и окончания, количество гостей. Система проверяет доступность столика на указанный временной слот.
- Просмотр своих бронирований — гость видит все свои бронирования с фильтрацией по статусу; администратор видит все бронирования системы.
- Подтверждение бронирования — администратор может изменить статус `pending` → `confirmed`.
- Отмена бронирования — гость может отменить бронирование со статусом `pending` или `confirmed` до даты начала. Администратор может отменить бронирование в любом статусе.
- Завершение бронирования — администратор переводит статус `confirmed` → `completed` по факту посещения.

- История бронирований — гость видит все свои бронирования с деталями ресторана и столика.

1.7. Отзывы (Reviews)

- Создание отзыва — гость может оставить отзыв только при наличии завершённого (completed) бронирования в данном ресторане. Один отзыв на одно бронирование.
- Просмотр отзывов — список отзывов по ресторану с оценкой, комментарием и датой.
- Удаление отзыва — администратор может удалить отзыв, нарушающий правила сервиса.

1.8. Административные функции

- Управление кухнями — CRUD для справочника cuisines (только администратор).
- Просмотр статистики — количество ресторанов, бронирований, средняя оценка по ресторанам (только администратор).

2. Нефункциональные требования

2.1. Производительность и масштабируемость

- Время ответа API — не более 200 мс для 95% запросов при стандартной нагрузке.
- Поддержка пагинации для всех списков (limit/offset).
- Кэширование часто запрашиваемых данных (список ресторанов, кухни, меню) через Redis (TTL 5–10 минут).
- Индексы БД на внешние ключи и поля фильтрации (city, avg_price_per_person, cuisine_id, booking_date).
- Горизонтальное масштабирование — бэкенд должен быть stateless (сессии не хранятся на сервере).

2.2. Безопасность

- Хэширование паролей — bcrypt (cost $\geq$ 10).
- JWT-токены с временем жизни: access token — 15 минут, refresh token — 7 дней.
- HTTPS на всех эндпоинтах.
- Защита от SQL-инъекций — использование параметризованных запросов (ORM/query builder).
- Валидация входных данных — на уровне схемы и бизнес-логики.
- CORS — разрешены только доверенные домены фронтенда.

- Rate limiting — не более 100 запросов в минуту с одного IP для неавторизованных и 200 — для авторизованных.
- Ролевая модель — middleware проверяет права доступа к ресурсам на каждый запрос.

2.3. Надёжность и отказоустойчивость

- Резервное копирование БД — ежедневное с retention 30 дней.
- Идемпотентность для критических операций (повторный запрос на создание бронирования не дублирует запись).
- Graceful degradation — при недоступности сторонних сервисов API возвращает корректные коды ошибок.

2.4. Наблюдаемость (Observability)

- Логирование всех запросов (method, path, status, duration) и критических операций (создание/отмена бронирования).
- Метрики — количество запросов, ошибок, времени ответа (Prometheus + Grafana).
- Correlation ID для каждого запроса (distributed tracing).

2.5. Документация

- OpenAPI (Swagger) — актуальная документация всех эндпоинтов с примерами запросов и ответов.
- README с инструкцией по запуску и описанием переменных окружения.

3. Выбор формата API

Для реализации выбран формат **REST API**. Сервис предполагает чёткое разделение ресурсов (рестораны, столики, бронирования, пользователи), что хорошо ложится на REST-семантику (GET/POST/PUT/PATCH/DELETE). Кроме того, REST обеспечивает лёгкую интеграцию с мобильными приложениями и сторонними клиентами через стандартную OpenAPI-спецификацию.

Вывод

Разработана спецификация REST API для сервиса бронирования столиков в ресторанах, полностью соответствующая схеме БД. API охватывает все ключевые сущности: пользователи, рестораны, столики, фотографии, меню, бронирования и отзывы. Определены функциональные требования к

каждому модулю системы, а нефункциональные требования обеспечивают производительность, безопасность и наблюдаемость сервиса в продакшн-среде.